Northeastern University

College of Engineering

EECE 2560: Fundamentals of Engineering Algorithms

Airport Code Lookup Service

Using a Binary Search Tree

Mini Project Report

Yasmein Asfour

Ayham Makhamra

Fall 2026

Contents

1 Abstract **2**

2 Introduction **2**

3 Objectives **2**

4 Program Design **3**

4.1 Main data structures 3

4.2 Loading the data 3

4.3 Searching and insertion 3

4.4 Deletion 4

4.5 Traversal, validation, and rebalancing 4

5 User Interface **4**

6 Testing **5**

6.1 Testing method 5

6.2 Automated test result 5

7 Results **6**

7.1 Original tree 6

7.2 Tree-shape experiment 6

8 Demonstration Videos **7**

9 Discussion **7**

10 Conclusion **7**

11 References **8**

A Reproducing the Test Run **8**

1 Abstract

In this project, we built an airport lookup program in C++17 using a binary search tree (BST). Each airport is stored by its three-letter IATA code. The program can load the OpenFlights data, find airports, add or remove records, print the airports in order, search within a range, and check that the tree is still valid. It can also save and reload the tree and rebuild it in a balanced shape. We tested all of the required operations, including the three deletion cases. We also compared different tree shapes to see how insertion order affects search speed.

2 Introduction

Airports usually have a three-letter IATA code, so the code works well as a key. In a BST, smaller codes go to the left and larger codes go to the right. This lets the program follow one path through the tree instead of checking every airport. How fast this works depends on the shape of the tree.

The main goal was to take the BST algorithms from class and use them in a working command-line program. The program also prints search paths, deletion cases, tree height, and average depth. These values helped us see what was happening inside the tree and made the program easier to test.

3 Objectives

For this project, we wanted the program to:

- Load valid airport records from the supplied OpenFlights data file.
- Store airports in a BST ordered by IATA code.
- Support searching, insertion, deletion, sorted listing, and range queries.
- Handle leaf, one-child, and two-child deletion cases correctly.
- Validate the BST using lower and upper key limits.
- Save a tree in preorder and rebuild the same shape later.
- Rebalance the tree using the middle element of a sorted list.
- Compare the shape and search performance of different insertion orders.

4 Program Design

4.1 Main data structures

The `Airport` class stores the information for one airport: IATA code, name, city, country, ICAO code, latitude, and longitude. The `AirportBST` class manages the tree. Each node stores one `Airport` and has pointers to a left child and a right child.

We use the IATA code as the key. When searching or inserting, the program compares the codes alphabetically. It goes left for a smaller code and right for a larger code. If the code is already in the tree, it is not added again.

Table 1: Main project files and their responsibilities.

File	Purpose
`src/Airport.h/.cpp`	Defines an airport record and its output format.
`src/AirportBST.h/.cpp`	Implements the BST operations and tree measurements.
`src/main.cpp`	Loads data and provides the command-line interface.
`tests/run_tests.sh`	Builds the program and checks the required behavior.
`docs/pseudocode.md`	Describes the main algorithms before implementation.

4.2 Loading the data

The loader reads the airport file one record at a time and handles commas inside quoted text. If the IATA field is `\N`, the record is skipped because there is no code to use as a key. The valid airports are inserted in the same order they appear in the file. There are 7,698 lines total. Out of those, 6,072 have valid IATA codes and 1,626 are skipped. The data did not contain any duplicate valid IATA codes.

4.3 Searching and insertion

Searching starts at the root. After each comparison, the program goes either left or right. It prints every code it visits so we can see the full search path. Insertion works in almost the same way, but it stops at an empty spot and puts the new node there.

Search and insertion take $O(h)$ time, where h is the height of the tree. In a balanced tree, the height is close to $\log_2 n$. In the worst case, the tree can turn into a long chain with height $n - 1$.

4.4 Deletion

To delete an airport, the program first searches for its code. Then it handles one of these three cases:

- **D1: leaf.** The node has no children and can be removed directly.
- **D2: one child.** The node is replaced by its only child.
- **D3: two children.** The smallest node in the right subtree is found. Its airport record replaces the deleted record, and the successor node is then removed.

These steps keep the tree in the correct order. The program prints D1, D2, or D3 so we can tell which case happened.

4.5 Traversal, validation, and rebalancing

An inorder traversal visits the left side, the current node, and then the right side. This prints all airport codes in alphabetical order. The range command uses the same idea, but it skips parts of the tree that cannot have a code in the requested range.

The validation function keeps a lower and upper limit for every recursive call. A node has to be between those limits. To rebalance the tree, the program first puts the records in sorted order. It picks the middle record as the root and repeats the same step on the left and right halves.

Table 2: Time complexity of the main operations.

Operation	Time	Reason
Search, insert, or delete	$O(h)$	At most one path from the root is followed.
Inorder listing	$O(n)$	Every node is visited once.
Range query	$O(h + k)$	Unneeded branches are pruned; k results are printed.
Validation	$O(n)$	Each node is checked once.
Balanced rebuild	$O(n)$	Each sorted record becomes one node.

5 User Interface

The program uses a simple text command loop. Some of the main commands are `load`, `find`, `add`, `remove`, `list`, `range`, `save`, `validate`, `rebalance`, and `experiment`. We used

commands because they make each BST operation easy to run and show during the demo.

The project can be built and started with:

```
g++ -std=c++17 -O2 src/*.cpp -o airports
./airports
```

After the program starts, the data is loaded with:

```
load data/airports.dat
```

6 Testing

6.1 Testing method

The test script builds the program and checks the main features. It tests loading, searching, adding, all deletion cases, range output, preorder save/reload, validation, and rebalancing. For each deletion case, the original tree is loaded again. This keeps one deletion from changing the setup for another test.

We used the following airport codes to check specific parts of the original tree:

Table 3: Representative test cases from the original file-order tree.

Test	Code	Expected result
Root	`GKA`	Found at the root
Leaf deletion	`AAC`	D1
Only left child	`GJT`	D2, child `GJR`
Only right child	`AAA`	D2, child `AAC`
Two children	`AEY`	D3, successor `AFA`
Deep search	`YZR`	Depth 177
Missing code	`XYZ`	Not found
Minimum and maximum	`AAA`, `ZZV`	Correct endpoints

6.2 Automated test result

Figure 1 shows the final test run. The script finished with the message `All Airport BST tests passed.`

```
(base) yasmeinasfour@Yasmeins-MacBook-Pro-595 YA-2M-Mini-Project-1 % make test
bash tests/run_tests.sh
All Airport BST tests passed.
(base) yasmeinasfour@Yasmeins-MacBook-Pro-595 YA-2M-Mini-Project-1 %
```

Figure 1: Terminal output from `make test`.

7 Results

7.1 Original tree

When we loaded the airports in file order, the tree had 6,072 nodes, a height of 177, and an average depth of 18.323. The root was `GKA`. Finding `BOS` took 12 comparisons and followed this path:

```
GKA -> AEY -> EGS -> BUJ -> BJA -> BSK -> BOY -> BNI
    -> BOH -> BOO -> BOU -> BOS
```

The range from `BOS` through `BWI` returned 113 airports and visited 127 nodes. It did not need to check all 6,072 nodes because it skipped branches outside the range.

After saving the original tree in preorder and loading it again, it had the same height and the same first three levels:

```
Level 0: GKA
Level 1: AEY MAG
Level 2: AAE EGS HGU POM
```

7.2 Tree-shape experiment

We built the tree three times using different insertion orders. Table 4 shows the results from our final run. The exact times can change a little depending on the computer.

Table 4: Effect of insertion order on tree shape and search time.

Build order	Height	Average depth	10,000 searches
File order	177	18.323	4.677 ms
Sorted order	6,071	3,035.500	316.810 ms
Balanced middle rule	12	10.653	3.061 ms

When the records were already sorted, every new code went to the right. This made the tree one long chain, so it had the worst height and search time. The balanced tree had a height close to $\log_2(6072)$ and was the fastest. The file-order tree worked much better than the sorted tree, but it was still taller than the balanced tree.

8 Demonstration Videos

We split the project demonstration into two videos:

- **Demo Part 1:** https://youtu.be/7mMD7evsa1c
- **Demo Part 2:** https://youtu.be/LFwiUs4Al4c?si=Xs-r1mi-l-U9g9WG

The links are clickable in the PDF.

9 Discussion

The biggest thing we learned was that the shape of the tree matters a lot. BST search is $O(h)$, not always $O(\log n)$, because the tree is not automatically balanced. The sorted-order test showed this clearly. The tree was still a valid BST, but it worked like a linked list and searches took much longer.

Deletion was the part that needed the most attention because the pointers have to be changed without losing part of the tree. Printing D1, D2, or D3 made it easier to check each case. Running validation after changes was also helpful because it checked the ordering of the whole tree.

One limitation is that the preorder save file only stores IATA codes. The original airport data has to be loaded first so the program can match those codes to the full records. This was enough for the assignment because the goal was to recreate the shape of the tree.

10 Conclusion

In the end, our program loaded the airport data and completed all of the required BST operations. The automated tests passed for searching, adding, every deletion case, traversal, ranges, validation, preorder loading, and rebalancing. The experiment also showed why balancing is important. The balanced tree was much shorter and faster than the tree made from sorted data. This project helped us understand how BST algorithms work in a larger program and how the tree's shape affects performance.

11 References

1. EECE 2560, *Mini Project: Airport Binary Search Tree*, Fall 2026.
2. OpenFlights, "Airport database," https://openflights.org/data.php.

A Reproducing the Test Run

From the project directory, run:

```
make test
```

The expected final line is:

```
All Airport BST tests passed.
```